

ساختار زمانی (۱۸۰ دقیقه)

زمان	بلوک	تمرکز
۱۰-۰	راه اندازی	بازسازی همان TrashNet پایپ لاین (بدون مفهوم جدید)
۴۰-۱۰	بلوک A	Degradation در برابر Overfitting + Residual
۹۰-۴۰	بلوک B	ساخت BasicResidualBlock از صفر + دیباگ shortcut
۱۰۰-۹۰	استراحت	—
۱۵-۱۰۰	بلوک C	ساخت TinyResNet، ردیابی shape، شمارش پارامتر، مشاهده ی گرادین
۱۸-۱۵۰	بلوک D	آموزش مقایسه ای + خواندن resnet18 + جمع بندی

بلوک ۰ - راه اندازی (۱۰ دقیقه)

این دقیقاً همان پایپ لاین کلاس صبح است - پایه ای است که روی آن کار در این جلسه کار می کنیم.

```
import random
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset, Subset
from torchvision import datasets, models, transforms
from pathlib import Path

SEED = 42
random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

```

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", DEVICE)

TRASHNET_ROOT = Path("data/trashnet")
TARGET_CLASSES = ["cardboard", "glass", "plastic"]

NORMALIZE = transforms.Normalize(mean=(0.5, 0.5, 0.5),
                                  std=(0.5, 0.5, 0.5))

image_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    NORMALIZE,
])

class TrashNetThreeClass(Dataset):
    def __init__(self, image_folder, target_classes):
        self.image_folder = image_folder
        self.classes = list(target_classes)
        self.class_to_idx = {name: i for i, name in enumerate(self.classes)}
        original_to_new = {
            image_folder.class_to_idx[name]: self.class_to_idx[name]
            for name in self.classes
        }
        self.indices = [
            Index for index, (_, original_label) in
            enumerate(image_folder.samples) if original_label in original_to_new
        ]
        self.original_to_new = original_to_new

    def __len__(self):
        return len(self.indices)

    def __getitem__(self, index):
        original_index = self.indices[index]
        image, original_label = self.image_folder[original_index]
        return image, self.original_to_new[original_label]

base_image_folder = datasets.ImageFolder(root=TRASHNET_ROOT,
transform=image_transform)
trashnet_dataset = TrashNetThreeClass(base_image_folder, TARGET_CLASSES)

generator = torch.Generator().manual_seed(SEED)
indices = torch.randperm(len(trashnet_dataset),
generator=generator).tolist()

```

```

split_point = int(0.8 * len(indices))
train_dataset = Subset(trashnet_dataset, indices[:split_point])
test_dataset = Subset(trashnet_dataset, indices[split_point:])

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True,
num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False,
num_workers=0)

class_names = trashnet_dataset.classes
images, labels = next(iter(train_loader))
print("شکل تصاویر:", images.shape)    # [32, 3, 64, 64]

```

موتور آموزش (داده‌شده – دوباره ننویسید، فقط استفاده کنید):

```

def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss, total_correct, total_examples = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        batch_size = labels.size(0)
        total_loss += loss.item() * batch_size
        total_correct += (logits.argmax(dim=1) == labels).sum().item()
        total_examples += batch_size
    return total_loss / total_examples, total_correct / total_examples

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, total_correct, total_examples = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        loss = criterion(logits, labels)
        batch_size = labels.size(0)
        total_loss += loss.item() * batch_size
        total_correct += (logits.argmax(dim=1) == labels).sum().item()
        total_examples += batch_size
    return total_loss / total_examples, total_correct / total_examples

```

چرا این دو تابع را دوباره نمی‌نویسیم؟ چون این دقیقاً همان «موتور آموزش شفاف» است که در Session 1 نوشتید و در Session 3 فایل صراحتاً می‌گوید باید همان را استفاده کنید. امروز تمرکز روی معماری است، نه حلقه‌ی آموزش.

بلوک A

Degradation در برابر Overfitting، و شهود ۳۰ - Residual دقیقه

A1 - تشخیص سریع (۸ دقیقه)

سه گزارش آزمایش زیر را در نظر بگیرید:

دقت تست (epoch آخر)	دقت ترین (epoch آخر)	آزمایش
۰.۵۵	۰.۹۵	۱
۰.۵۸	۰.۶۰	۲ (مدل 20 لایه)
۰.۴۴	۰.۴۵	۳ (همان مدل ۲۰ لایه، اضافه‌شدن ۱۰ لایه‌ی دیگر بدون residual)

سؤال: کدام آزمایش نمونه‌ی Overfitting است و کدام نمونه‌ی Degradation؟ برای هرکدام یک جمله دلیل بیاورید.

A2 - کدنویسی: شهود Residual با یک مثال (۱۲ دقیقه)

در این سؤال قرار است خودتان تفاوت یک شبکه‌ی plain و یک شاخه‌ی residual را در ساده‌ترین حالت ممکن پیاده‌سازی کنید.

ورودی و هدف را به‌صورت زیر در نظر بگیرید:

```
torch.manual_seed(42)
inputs = torch.linspace(-2, 2, 100).unsqueeze(1)
targets = inputs.clone() # هدف: یادگیری تابع هویت  $f(x) = x$ 
```

ساختار مدل plain را در اختیار دارید:

```
plain_identity = nn.Sequential(nn.Linear(1, 16), nn.ReLU(),
                               nn.Linear(16, 1)).to(DEVICE)
```

اکنون باید خودتان شاخه‌ی residual را با دقیقاً همان معماری بسازید:

```
# TODO: residual_branch کنید
```

سپس optimizerهای مربوط به دو مدل و $\text{criterion} = \text{nn.MSELoss}$ را تعریف کنید و یک حلقه آموزش ۱۰۰ مرحله‌ای بنویسید.

در بخش plain، خروجی مدل را مستقیماً با هدف مقایسه کنید:

```
in_identity(inputs_device)
```

اما در بخش residual، باید ساختار زیر را خودتان در کد بنویسید:

```
# TODO: بسازید  $F(x) + x$  را به صورت residual خروجی
```

سپس Loss هر دو مدل را در هر مرحله ذخیره کنید و در پایان Loss اولیه و Loss نهایی هر دو مدل را چاپ کنید.

سؤالهای تحلیلی:

سؤال 1: کدام مدل Loss اولیه‌ی کمتری داشت؟ چرا؟

سؤال 2: اگر شاخه residual دقیقاً خروجی صفر تولید کند، خروجی کل بلوک چه می‌شود؟

سؤال 3: این رفتار را با رابطه زیر مرتبط کنید:

$$H(x) = F(x) + x$$

A3 - کوتاه: چرا این مهم است؟ (۱۰ دقیقه)

سؤال: فرض کنید یک شبکه عمیق دارید که ۵ لایه آخر آن عملاً هیچ تبدیلی ایجاد نمی‌کنند و خروجی مطلوب همان چیزی است که قبل از این ۵ لایه وجود داشته است.

طبق شهود بخش 2A، اگر این ۵ لایه به صورت بلوک‌های residual ساخته شده باشند، مدل در مقایسه با ۵ لایه plain چگونه می‌تواند راحت‌تر این لایه‌ها را «نادیده بگیرد»؟

پاسخ باید کاملاً مفهومی باشد و نیازی به محاسبه ریاضی ندارد.

بلوک B

ساخت BasicResidualBlock از صفر (۵۰ دقیقه)

B1 - طراحی روی کاغذ (۱۰ دقیقه)

مشخصات زیر را بدون نوشتن کد بررسی کنید:

شاخه اصلی:

Conv2d(in→out, k=3, stride=s, padding=1) → BatchNorm2d → ReLU →
Conv2d(out→out, k=3, stride=1, padding=1) → BatchNorm2d

شاخه shortcut:

- $\rightarrow \text{nn.Identity}()$ باشد $\text{in_channels} == \text{out_channels}$ و $\text{stride}=1$ اگر
- $\rightarrow \text{Conv2d}(\text{in} \rightarrow \text{out}, k=1, \text{stride}=s) + \text{BatchNorm2d}$ در غیر این صورت

خروجی نهایی:

جمع دو شاخه، سپس ReLU.

سؤال: اگر ورودی شکل [32, 32, 16, 32] داشته باشد و $\text{BasicResidualBlock}(16, 32, \text{stride}=2)$ بسازیم:

۱. شکل خروجی شاخه اصلی چیست؟

۲. شکل خروجی shortcut باید چه باشد تا جمع ممکن شود؟

۳. آیا این جا shortcut باید Identity باشد یا Projection؟ چرا؟

B2 - کدنویسی از صفر: BasicResidualBlock

طبق مشخصات B1، کلاس زیر را کاملاً خودتان پیاده‌سازی کنید و تا حد ممکن بدون نگاه کردن به فایل مرجع پیش بروید:

```
class BasicResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()

# TODO: conv1, bn1, relu, conv2, bn2
# TODO: self.shortcut را به صورت Identity یا Projection بسازید

    def forward(self, x):
        # TODO: identity = self.shortcut(x)
```

```
# TODO: از شاخه‌ی اصلی x عبور
# TODO: جمع دو شاخه
# TODO: نهایی ReLU
```

سناریوی تست

دو تست متفاوت طراحی کنید که هر دو حالت زیر را پوشش دهند:

- حالتی که shortcut باید Identity باشد.
- حالتی که shortcut باید Projection باشد.

برای هر تست، خودتان مشخص کنید:

1. in_channels
2. out_channels
3. stride
4. اندازه batch
5. اندازه فضای ورودی
6. شکل خروجی مورد انتظار

سپس تست‌ها را اجرا کنید و نشان دهید که شکل خروجی با پیش‌بینی شما مطابقت دارد.

برای راهنمایی بیشتر:

```
block_same = BasicResidualBlock(?)
block_downsample = BasicResidualBlock(?)

test_input = torch.randn(?)
print("identity shortcut:", block_same().shape)
# انتظار: [4, 16, 32, 32]
print("projection shortcut:", block_downsample().shape)
# انتظار: [4, 32, 16, 16]
```

B3 - دیباگ: shortcut ناسازگار (۱۲ دقیقه)

کد زیر عمداً دارای خطای shape است:

```
class ShapeMismatchBlock(nn.Module):
    def __init__(self):
        super().__init__()
```

```

        self.main = nn.Conv2d(16, 32, kernel_size=3, stride=2,
padding=1)
        self.activation = nn.ReLU()

    def forward(self, x):
        return self.activation(self.main(x) + x)

```

سؤال (بدون اجرا، فقط استدلال):

1. اگر x شکل $[32, 32, 16, 32]$ داشته باشد، شکل $\text{self.main}(x)$ چیست؟
2. چرا عبارت $x + \text{self.main}(x)$ خطا می‌دهد؟
3. shortcut باید چه تغییری کند تا جمع دو شاخه ممکن شود؟
4. کد اصلاح‌شده را بنویسید. سپس نسخهٔ اصلاح‌شده را اجرا و تست کنید.

B4 - گروهی: کی Identity، کی Projection؟ (۸ دقیقه)

برای هر سناریو مشخص کنید shortcut باید Identity باشد یا Projection، و اگر Projection است، پارامترهای دقیق Conv2d آن را بنویسید:

۱. BasicResidualBlock(32, 32, stride=1)
۲. BasicResidualBlock(64, 64, stride=2)
۳. BasicResidualBlock(16, 64, stride=1)

بلوک C

ساخت TinyResNet، ردیابی Shape، مشاهده‌ی گرادین (۵۰ دقیقه)

C1 - کدنویسی از صفر: ۲۰ (TinyResNet دقیقه)

طبق مشخصات زیر، کلاس TinyResNet را با استفاده از BasicResidualBlock ای که خودتان در بخش 2B نوشته‌اید، پیاده‌سازی کنید:

- **stem:**
Conv2d(3→16, k=3, padding=1, bias=False) → BatchNorm2d(16) → ReLU
- **stage1:**
دو BasicResidualBlock(16, 16, stride=1)
- **stage2:**
سپس BasicResidualBlock(16, 32, stride=2)
BasicResidualBlock(32, 32, stride=1)
- **stage3:**
سپس BasicResidualBlock(32, 64, stride=2)
BasicResidualBlock(64, 64, stride=1)
- **head:**
AdaptiveAvgPool2d((1,1)) → Flatten → Linear(64, num_classes)

قالب اولیه:

```
class TinyResNet(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        # TODO: stem
        # TODO: stage1
        # TODO: stage2
        # TODO: stage3
        # TODO: head
    def forward(self, x):
        # TODO
```

تذکر: forward باید داده را به‌ترتیب از stage3، stage2، stage1، stem و در پایان head عبور دهد.

سؤال مفهومی:

چرا از $(1,1)$ AdaptiveAvgPool2d به جای Flatten مستقیم استفاده شده است؟ این مفهوم را کجا قبلاً دیده‌اید؟

راهنما: به مفهوم Global Average Pooling در Session 2 فکر کنید.

C2 - ردیابی Shape واقعی (۱۰ دقیقه)

```
model = TinyResNet(num_classes=len(class_names)).to(DEVICE)

with torch.no_grad():
    x = images.to(DEVICE)
    x = model.stem(x); print("stem بعد از:", x.shape)
    x = model.stage1(x); print("stage1 بعد از:", x.shape)
    x = model.stage2(x); print("stage2 بعد از:", x.shape)
    x = model.stage3(x); print("stage3 بعد از:", x.shape)
    logits = model.head(x); print("logits:", logits.shape)
```

سؤال: قبل از اجرا، روی کاغذ پیش‌بینی کنید شکل تنسور بعد از هر مرحله چیست (ورودی اینجا [32,3,64,64] است؛ چون اندازه‌ی batch برابر ۳۲ و ابعاد تصاویر ۶۴×۶۴ است). سپس با خروجی واقعی تطبیق دهید.

C3 - شمارش پارامتر (۵ دقیقه)

```
def count_parameters(model):
    return sum(p.?.() for p in model.?.() if p.requires_grad)

print("تعداد پارامتر TinyResNet:", count_parameters(model))
```

سؤال: اگر یک هم‌گروهی گفت «این مدل بیشتر از یک CNN ساده پارامتر دارد، پس حتماً بهتر است»، طبق محتوای تدریس شده چه پاسخی می‌دهید؟

C4 - مشاهده‌ی گرادیان (کد داده‌شده، فقط اجرا و تفسیر) (۱۵ دقیقه)

```
class PlainConvStack(nn.Module):
    def __init__(self, channels=(16, 16, 16, 16, 16, 16)):
        super().__init__()
        layers, in_channels = [], 3
        for out_channels in channels:
            layers += [nn.Conv2d(in_channels, out_channels, 3,
padding=1, bias=False),
```

```

        nn.BatchNorm2d(out_channels), nn.ReLU())]
    in_channels = out_channels
    self.features = nn.Sequential(*layers)
    self.head = nn.Sequential(nn.AdaptiveAvgPool2d((1,1)),
nn.Flatten(), nn.Linear(in_channels, len(class_names)))

    def forward(self, x):
        return self.head(self.features(x))

def one_batch_gradient_report(model, loader, device):
    model = model.to(device); model.train()
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
    batch_images, batch_labels = next(iter(loader))
    batch_images, batch_labels = batch_images.to(device),
batch_labels.to(device)
    optimizer.zero_grad()
    loss = criterion(model(batch_images), batch_labels)
    loss.backward()
    grad_norm = next(p.grad.norm().item() for p in
model.parameters() if p.grad is not None)
    return {"loss": loss.item(), "first_parameter_grad_norm":
grad_norm}

torch.manual_seed(42)
print("Plain:", one_batch_gradient_report(PlainConvStack(),
train_loader, DEVICE))
torch.manual_seed(42)
print("TinyResNet:",
one_batch_gradient_report(TinyResNet(num_classes=len(class_names)),
train_loader, DEVICE))

```

سؤال: با نتیجه‌ی واقعی خودتان (نه فرضی)، norm گرادیان کدام مدل بزرگ‌تر بود؟

⚠ **نکته‌ی مهم:** این فقط گزارش یک batch است، نه اثبات قطعی که یک معماری همیشه گرادیان بهتری دارد – اگر نتیجه‌ی شما برخلاف انتظار بود (مثلاً plain گرادیان بزرگ‌تری داشت)، این خودش یک نتیجه‌ی معتبر و قابل بحث است، نه یک خطا.

بلوک D

آموزش مقایسه‌ای، خواندن resnet18، و جمع‌بندی (۳۰ دقیقه)

D1 - آموزش کوتاه هر دو معماری (۱۰ دقیقه)

```
def run_short_experiment(model, name, epochs=2):
    torch.manual_seed(42)
    model = model.to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    history = []
    for epoch in range(epochs):
        train_loss, train_acc = train_epoch(model, train_loader,
criterion, optimizer, DEVICE)
        test_loss, test_acc = evaluate(model, test_loader,
criterion, DEVICE)
        history.append({"epoch": epoch+1, "train_acc": train_acc,
"test_acc": test_acc})
        print(f"{name:>10} | epoch {epoch+1} | train
acc={train_acc:.3f} | test acc={test_acc:.3f}")
    return model, history

plain_trained, plain_history =
run_short_experiment(PlainConvStack(), "plain")
resnet_trained, resnet_history =
run_short_experiment(TinyResNet(num_classes=len(class_names)),
"residual")
```

سؤال: طبق فایل، چرا نمی‌توانیم از نتیجه‌ی همین دو خط بگوییم «ResNet همیشه برنده است»؟ حداقل دو محدودیت این آزمایش را نام ببرید. (راهنما: تعداد epoch، تعداد split تصادفی).

D2 - خواندن معماری واقعی ۱۰ resnet18 (دقیقه)

```
resnet18 = models.resnet18(weights=None,
num_classes=len(class_names))
resnet18.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,
padding=1, bias=False)
resnet18.maxpool = nn.Identity()
resnet18 = resnet18.to(DEVICE)
```

```
print(resnet18)
```

سؤال: در خروجی چاپ شده، دنبال این سه چیز بگردید و پیدا کنید: ۱. کلمه‌ی BasicBlock – همان‌گویی است که خودتان در بلوک B ساختید. ۲. کلمه‌ی downsample – این دقیقاً همان shortcut نوع Projection شماسست، فقط با اسم دیگر. ۳. weights=None یعنی چه؟ چرا امروز عمداً این‌طور تنظیم شده؟

D3 - جمع‌بندی (۱۰ دقیقه)

۱. تفاوت $H(x)$ و $F(x)$ در یک بلوک residual چیست؟
۲. چرا وقتی تبدیل مطلوب نزدیک به هویت است، shortcut کمک می‌کند؟
۳. کی به‌جای $\text{nn.Identity}()$ به یک shortcut از نوع Projection نیاز داریم؟
۴. چرا Degradation دقیقاً همان چیزی نیست که Overfitting نامیده می‌شود؟
۵. با نتایج واقعی D1 خودتان: کدام مدل امروز بهتر عمل کرد؟ آیا این نتیجه با شهود بخش A2 همخوانی داشت؟